CosmoTrekk

Your Journey Has No Horizon.

Junior Astronaut Mission Trainer

Presenter & Developer Guide

Everything a first-year B.Tech student needs to understand, run, deploy and demo the project, mostly as diagrams.

React + TypeScript | Phaser 3 game engine | Vite | PWA (offline) | Vercel (free)

Optional: Gemini · NASA · Supabase



Pages 2–3
What the game is and how a turn works

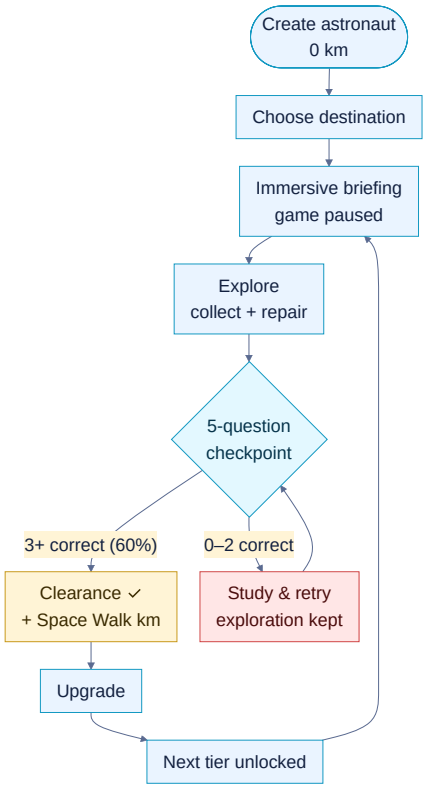
Pages 4–11
Tech stack, architecture, code walkthrough

Pages 12–14
Setup, deploy, 3-minute demo, judges' FAQ

1 CosmoTrekk in 60 seconds

A family-friendly space-science adventure. Players **explore** a destination, **repair** equipment, pass a **5-question checkpoint**, and earn **Space Walk** kilometres. Goal: *"Explore further, grow your knowledge, and build your Space Walk."*

The core loop (one "tier")



Checkpoint rules
- Exactly 5 questions, 4 options each
- Each correct answer = 20%
- **3 of 5 (60%) passes**
- A wrong answer never ends the attempt
- Unlimited retries, fresh questions

Space Walk rewards
Tier 1 → +100 km
Tier 2 → +250 km
Tier 3 → +500 km
Tier 4 → +1,000 km
Full expedition = 1,850 km
Game distance, not real travel.

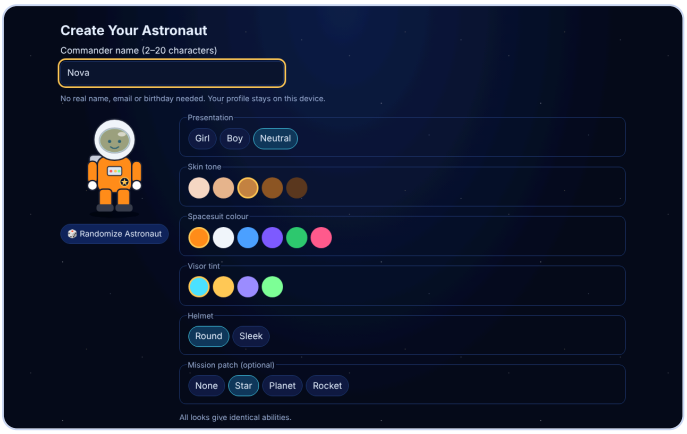
Never gives km for…
- picking a faraway planet
- idling or refreshing
- reading or practice quizzes
- replaying a cleared tier
- editing saved scores

What can you play?

Mode	Content
8 starting expeditions 4 tiers each	Mars (planet) · Moon · Sun (star, remote deck only) · Asteroid KT-7* · Comet Lumen-K* · Exoplanet Aurelia b* · Relay-9 satellite* · Horizon Research Station* (* = clearly labelled fictional training sites)
Featured campaign 4 story levels	Earth → Moon *(Lunar Readiness)* → Mars *(Red Planet Pioneer, "Chapter One Complete")* → Venus *(Heat Shield Protocol)* → Saturn Orbital Station *(Ring Guardian: energy puzzle + friendly drone → Horizon Star Map)*
Know Your Space	44 study cards across 4 sections, 3 reading levels, 6 comparisons, unranked practice quizzes, bookmarks, search
Space Walk Rankings	"On this device" + labelled "Demo players"; optional server-verified board

Honesty rule: the game labels **Science Fact** (sourced) vs **Simulation Fiction** (stations, drones, star map). Diagrams say *"Illustrative · Not to scale."* Real distances always name a reference point.

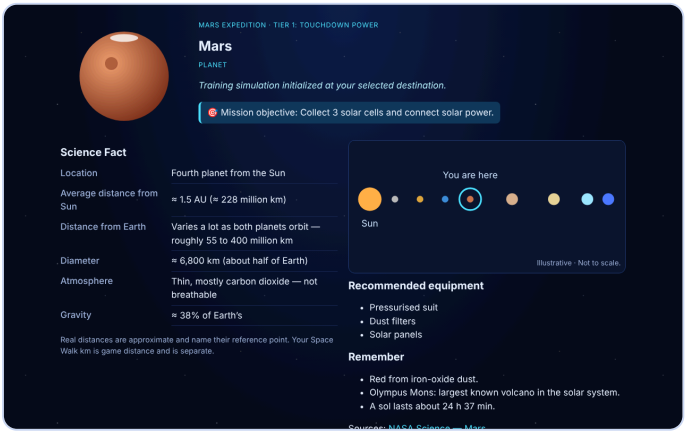
2 What the player sees



① Create astronaut: name, look, live preview. Starts at 0 km.



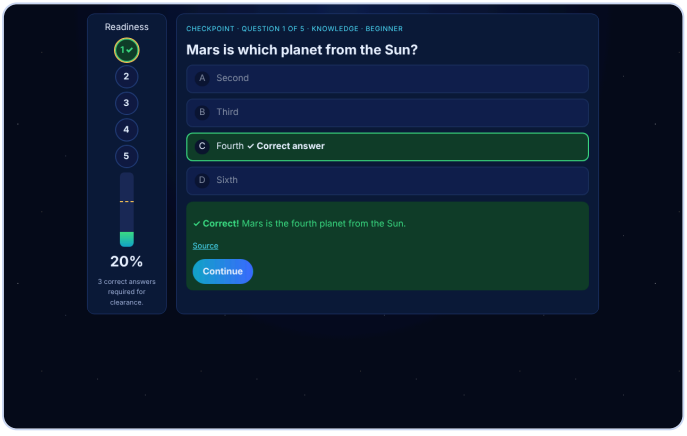
② Mission map: campaign + 8 expeditions, tiers unlock one by one.



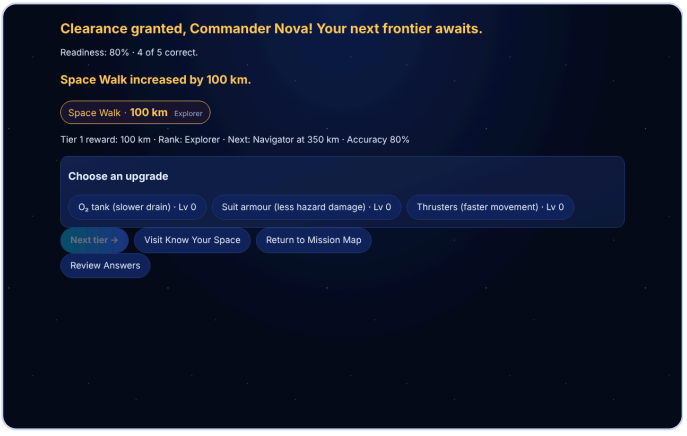
③ Briefing: sourced facts, diagram, equipment. Game paused.



④ Exploration (Phaser): collect items, avoid hazards, press E.



⑤ Checkpoint: answer locks, ✓/× + explanation + source.



⑥ Results: clearance, km awarded once, choose an upgrade.

Controls

W A S D / ← ↑ → ↓ move · E / Space interact · Esc pause

Phones: on-screen pad + big ✋ button

HUD

Space Walk · Oxygen · Fuel · Health · Suit · Modules · Objective · Pause · Sound. Oxygen refills on the map. Running out just restarts the activity.

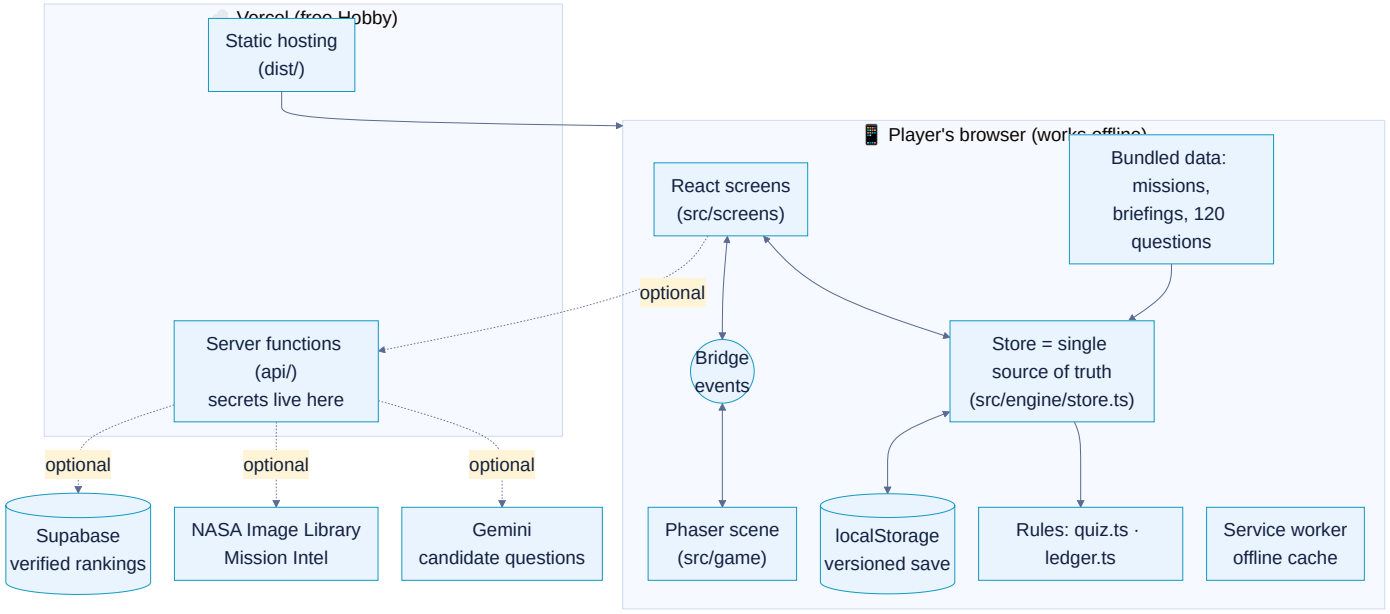
3 The tech stack, explained simply

Every tool is **free**. If you know basic HTML/JS, think of it like this:

Tool	What it is	Simple analogy	Where in the code
TypeScript	JavaScript + "types" that catch mistakes before running	Spell-check for code	every `.ts / .tsx` file; shapes in `src/types.ts`
React	Builds the UI from small reusable components	LEGO bricks for screens	`src/screens/`, `src/components/`
Phaser 3	2D game engine: draws ~60 frames/sec, keyboard, camera	The "game world" stage	`src/game/ExplorationScene.ts`
Vite	Dev server + bundler (turns TS into fast JS)	The kitchen that cooks the code	`vite.config.ts`, `npm run dev/build`
Zod	Checks data has the right shape at runtime	Security guard checking IDs	`save.ts`, `questionSchema.ts`
localStorage	Browser storage for the save file	The game's notebook	`src/engine/save.ts`
PWA + Service Worker	Makes the site installable and offline	A downloaded copy of the website	`vite.config.ts` (VitePWA)
Web Audio API	Generates music and sound effects in code	A tiny synthesizer	`src/audio/audio.ts`
Vitest	Runs automatic tests	A robot examiner	`tests/`, `npm test`
Git + GitHub	Version history + online code storage	Google Drive with undo history	`.gitignore`
Vercel	Free hosting + "serverless functions"	The shop window on the internet	`vercel.json`, `api/`
Serverless function	Small server code that runs only when called	A waiter who appears when you ring	`api/*.ts`
Gemini (optional)	Google AI. Drafts *candidate* questions from approved facts	An assistant writing drafts for a teacher	`api/generate-questions.ts`
NASA Image Library (optional)	Free NASA media search	NASA's photo album	`api/intel.ts`
Supabase (optional)	Free Postgres database + guest login	An online scoreboard with a referee	`supabase/*.sql`, `api/ranked/`

Key idea for judges: the **whole game runs in the browser** with zero keys and zero servers. Gemini, NASA and Supabase are **optional add-ons** that fail gracefully back to the bundled, reviewed content.

4 Architecture: how the pieces connect



Inside the browser

- **React screens** show menus, HUD, quiz, results.
- **Phaser** draws the playable world only.
- They talk through the **Bridge** (named messages), never directly. That keeps them cleanly separated.
- The **Store** holds all state and every action (`startStage`, `answer`, `finishQuiz`…).
- **Rules** live in tiny pure files that are easy to test.

On Vercel

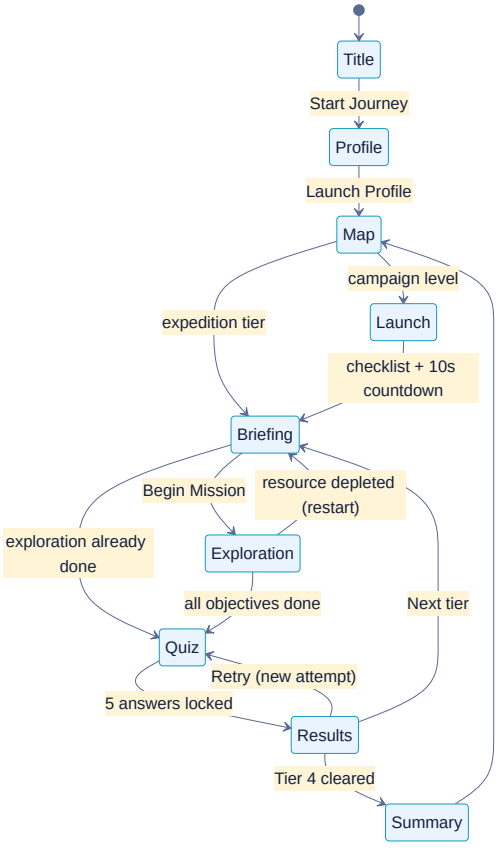
- **Static hosting** serves the built game (`dist/`).
- **Server functions** in `api/` hold secrets (Gemini key, Supabase service key). Secrets **never** reach the browser.
- Only variables starting with `VITE_` are public.
- API calls are never cached offline (service worker rule: `NetworkOnly`).

Folder map

```
cosmotrekk/
├ src/
│  ├ main.tsx · App.tsx      ← entry point + screen router, timers, audio unlock, PWA banner
│  ├ types.ts                ← all data shapes (Mission, Question, Profile…)
│  ├ config/rewards.ts       ← 100/250/500/1000 km, ranks, 3-of-5 rule
│  ├ data/                   ← missions, briefings, catalogue (44 cards), 120 questions, sources
│  ├ engine/                 ← quiz.ts · ledger.ts · save.ts · store.ts · questionSchema.ts · connected.ts
│  ├ game/                   ← ExplorationScene.ts (one scene for ALL stages) · PhaserGame.tsx · bridge.ts
│  ├ screens/                ← Title, Profile, MissionMap, Briefing, Explore, Quiz, Study, Rankings, Settings
│  ├ components/             ← Avatar, Art (planets & diagrams), Hud, RankedChallenge
│  └ audio/audio.ts          ← synthesized music & sound effects
├ api/                       ← optional Vercel server functions
├ supabase/                  ← optional SQL (tables, row-level security, functions, seed)
├ tests/                     ← 17 automatic tests
└ README.md + 7 other docs   ← setup, scoring, demo script, team tasks…
```

5 The game as a state machine

The game is always on exactly **one screen** (a "state"). Buttons trigger **transitions**. All of this lives in `src/engine/store.ts`.



How a React screen reads state

```
const s = useGame();          // subscribe to the store
const p = activeProfile(s);   // current commander
<SpaceWalkChip km={p.spaceWalkKm} />  // re-renders on change
```

How a button changes state

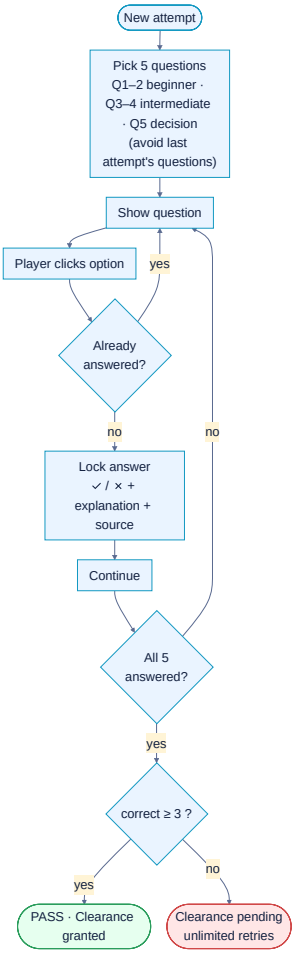
```
<button onClick={() => startStage('exp-mars', 1)}>
// store.ts → sets run = {mission, tier}
// → go('briefing') → App shows BriefingScreen
```

Why this design?

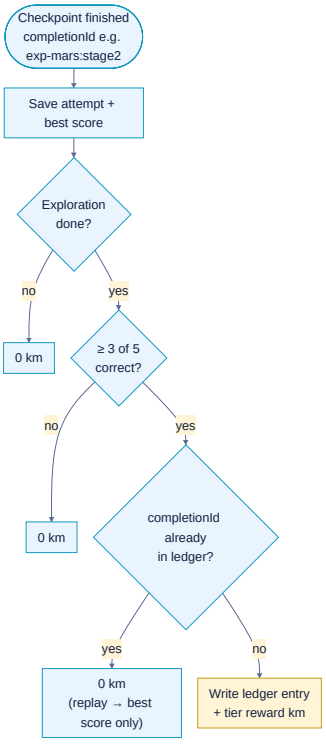
Problem	Solution in CosmoTrekk
Quiz failed. Must the player explore again?	No. `profile.explored` remembers it, and the briefing offers **"Go to Checkpoint"**.
Tab hidden or pause pressed during exploration	`visibilitychange` → pause menu → Bridge `pause` → oxygen and timers freeze.
Oxygen runs out	Friendly dialog → `restartActivity()`. Km and campaign are untouched.
Active play time must exclude paused time	`App.tsx` adds 1 s only when the screen is explore/quiz/launch, not paused, and the tab is visible.

6 Checkpoint & reward logic (the heart of the game)

Checkpoint flow — `src/engine/quiz.ts`



Reward ledger — `src/engine/ledger.ts`



"Idempotent" = doing it twice has the same effect as doing it once. Each milestone ID (e.g. `exp-mars:stage2`) can pay km **only once**, ever. On reload, total km is **recalculated from the ledger**, so editing the save file does nothing.

The real code (with its line comments)

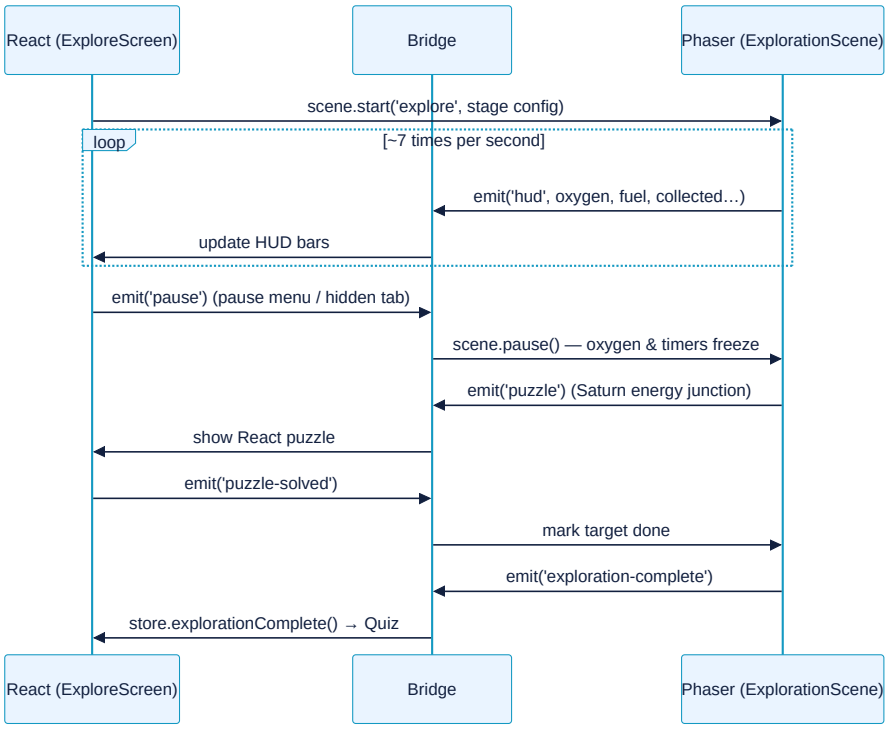
```
export function hasPassed(correct: number): boolean { // the clearance rule
  return correct >= PASS_CORRECT; // 3/5 (60%) or more passes; 0, 1, 2 do not
}
// inside applyCheckpoint(...) in ledger.ts:
if (!p.explored.includes(id)) return none;  // BOTH exploration AND quiz required
if (!passed) return none;                   // below 60% → no distance
if (p.ledger.some(e => e.completionId === id)) return none; // replay cannot farm distance
const km = TIER_REWARD_KM[tier];            // same schedule for every category
```

Answer locking
`submitAnswer()` ignores a second click on the same question, so double-clicks can't change or double-count.

Grade once
`finishQuiz()` has a `finishing` guard, so a double "See result" can't award twice.

Retry = new attempt
`newAttempt()` starts with empty answers and avoids the last questions. Scores never combine.

7 React ⇄ Phaser bridge, and saving progress

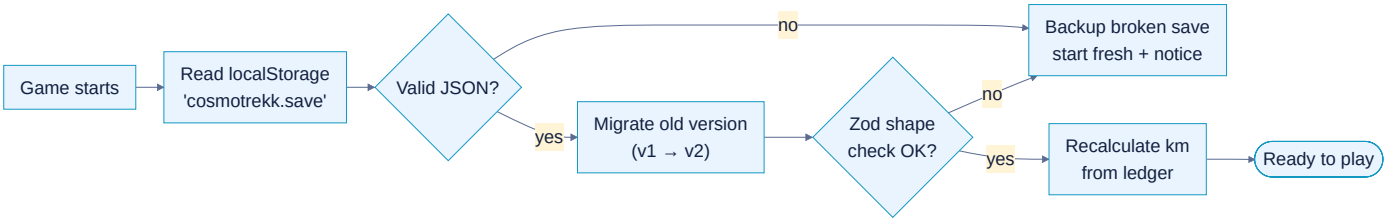


One scene plays every stage. `ExplorationScene` reads a **stage config** from `missions.ts`: theme, collectible (label, count, colour), ordered targets (normal / puzzle / drone), hazards, floating or ground. Adding a new mission = adding data, not new game code.

Only one Phaser game exists. It lives in a permanent `#phaser-root` div, is hidden and put to sleep between missions, and is reused next time.

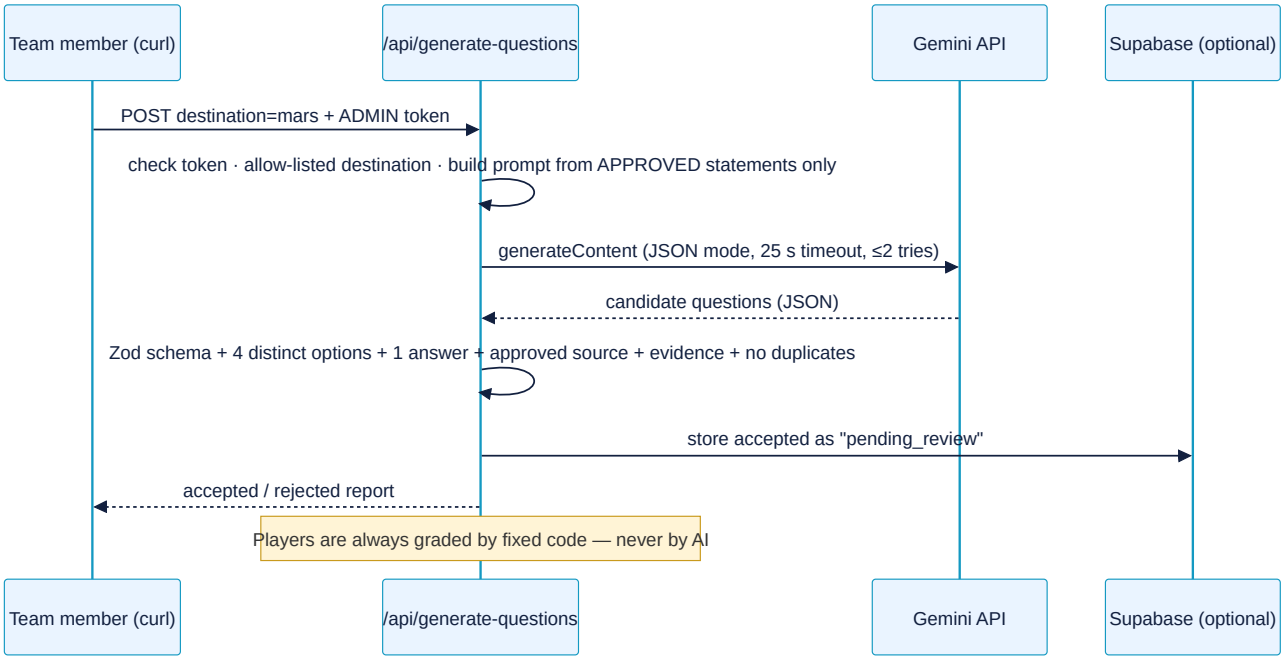
All art is drawn in code (shapes and SVG), so there are no image files to licence, it works offline, and it loads fast. Each frame, `update()`: reads input → moves astronaut → drains oxygen/fuel → moves hazards → checks pickups → checks the active target → sends HUD data.

Saving — `src/engine/save.ts`



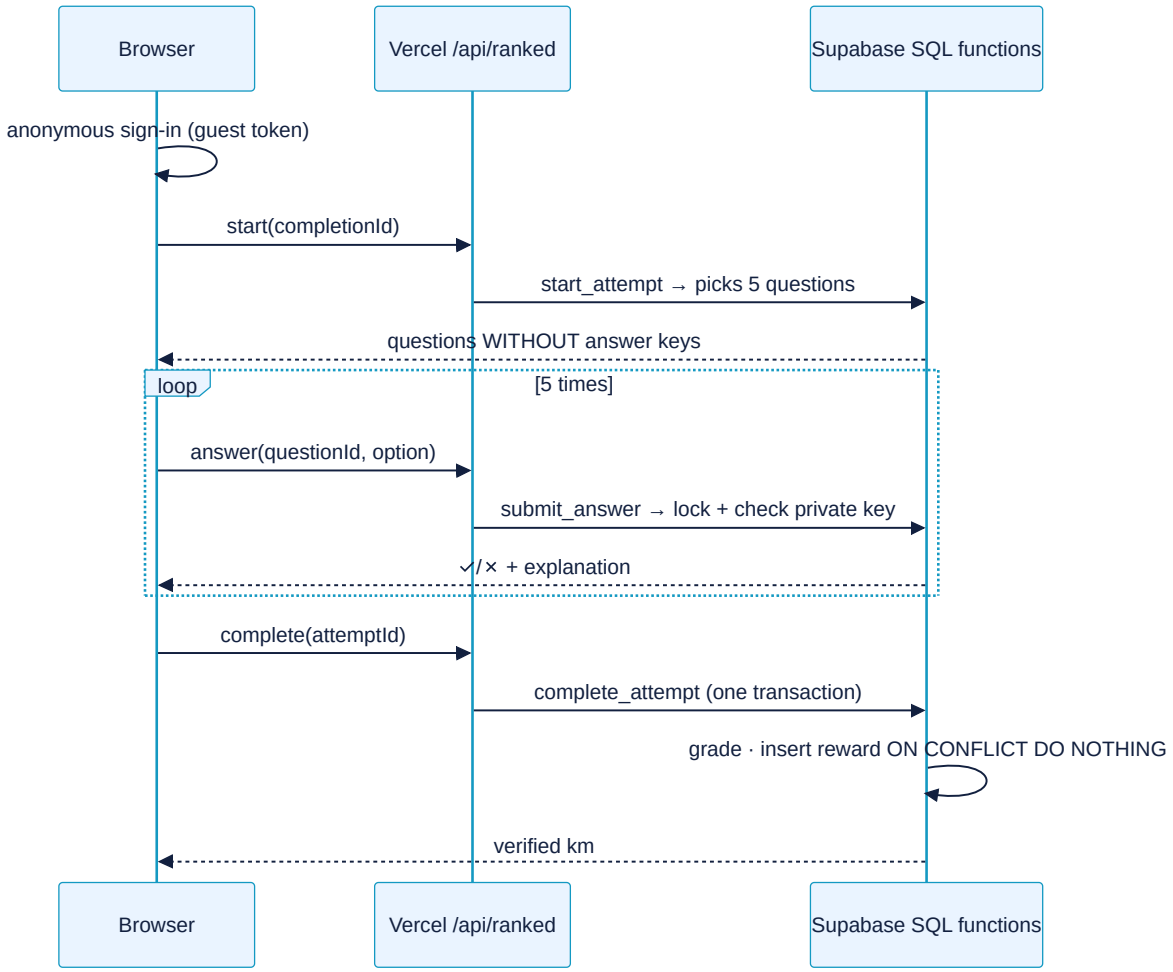

```
{ "version": 2, "activeProfileId": "p-…", "settings": { "master": true, "demoMode": false, … },
  "profiles": [ { "name": "Nova", "spaceWalkKm": 350, "ledger": [ { "completionId": "exp-moon:stage1",
"km": 100, … } ],
                "explored": [ … ], "attempts": [ … ], "bestScores": { … }, "upgrades": { … },
"badges": [ … ] } ] }
```

8 Optional AI: Gemini drafts questions, humans approve



Feature	How it stays safe and honest
Gemini (free key from Google AI Studio)	Protected by `ADMIN_REFRESH_TOKEN`. Only allow-listed destinations, and only approved source statements go into the prompt. Strict schema check plus a check that the evidence matches an approved statement. Results are stored as **pending review**. **Schema-valid ≠ factually true**, so a human approves. Timeouts, quota (429) and invalid JSON all fall back to the reviewed bank.
NASA Mission Intel (no key)	Free NASA Image Library. Allow-listed topics, 8 s timeout, 1-day CDN cache. Labels are honest: *"Cached source content · checked at…"* or *"Offline training bank"*. Never "Live".

9 Optional verified rankings (Supabase, free plan)



Topic	Explanation
Security	Row-level security on every table. Browsers can only **read** the leaderboard. Questions go out without answer keys. Grading and km happen in SQL functions inside one transaction. Primary key `(user_id, completion_id)` = once only. Demo mode is refused.
Public vs secret keys	`VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` are public (safe in the browser). `SUPABASE_SERVICE_ROLE_KEY` is **secret** and lives only in Vercel server functions.
Limitations	No browser game is cheat-proof. Exploration is still client-side. If the ranked pool reuses bundled questions, answers are visible in the client. Use new teacher-approved questions for a real contest.

Tested so far: Gemini, NASA and Supabase were tested with **mocked** responses and load correctly as Node modules. They were **not** tested against the live services. Add your free keys and test before demo day (`API_SETUP.md`). The offline game itself was fully tested in the browser.

10 How to read any file in 2 minutes

Every code file has a **header comment** saying what it does, plus a **comment beside almost every line**. JSON files (`package.json`, `vercel.json`) can't contain comments, so they're explained here.

Example: a stage is just data (`src/data/missions.ts`)

```
s(2, 'Relay Repair', 'moon', 'moon',                          // tier, title, destination
(briefing+questions), art theme
  'Collect 4 repair modules and repair the communication relay.', // objective shown in the HUD
  { label: 'Repair module', count: 4, color: GOLD },          // what to collect, how many, colour
  [{ label: 'Comms relay', action: 'Repair relay', needs: 4 }], // target(s) in order; needs = parts
required
  'Micrometeoroid'),                                           // hazard name (tier 2 → 2 hazards)
```

Example: one question (`src/data/questions.ts`)

```
['intermediate', 'Why do we always see the same side of the Moon from Earth?',     // difficulty,
question
 ['The Moon does not spin', 'It is tidally locked: it spins once per orbit',        // 4 options (a,
b, c, d)
  'Earth blocks the other side', 'The far side is always dark'],
 'b', 'Tidal locking means one rotation takes the same time as one orbit.', ['tidal-locking']], //
answer, explanation, tags
```

The JSON files explained

File	What each part means
`package.json`	**name** cosmotrekk · **scripts**: `dev` (start), `build` (typecheck + build), `preview`, `test`, `typecheck` · **dependencies**: react, react-dom, phaser, zod · **devDependencies**: tools used only while building (vite, typescript, vitest, PWA plugin).
`package-lock.json`	Exact library versions, so everyone installs the same thing. Don't edit by hand.
`vercel.json`	Tells Vercel: framework = Vite, build = `npm run build`, output = `dist`, and send unknown page URLs to `index.html` (but not `/api`).
`tsconfig.json`	TypeScript settings. **strict** mode on = maximum mistake-catching.
`.env.example`	Blank list of the optional keys. Copy it to `.env.local` or paste the values into Vercel. Never commit real keys.

Where to change things

Reward km / ranks → `config/rewards.ts`

Add a question → `data/questions.ts` then `npm test`

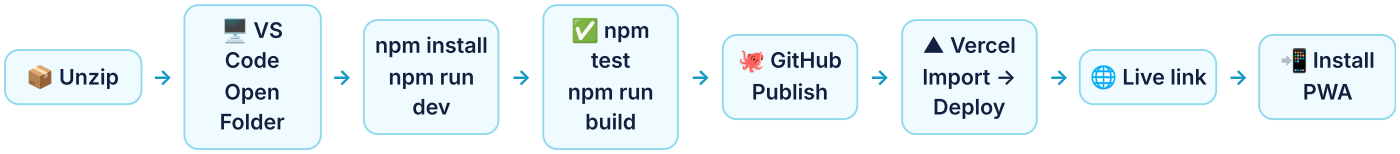
New study card → `data/catalogue.ts`

Drain speed / hazards → `game/ExplorationScene.ts`

Colours / layout → `styles.css`

New screen → `screens/` + register in `App.tsx`

11 From ZIP to a live link (step by step, all free)



A. One-time installs

1. **Node.js LTS**: nodejs.org → LTS → install
2. **Git**: git-scm.com → install
3. **VS Code**: code.visualstudio.com
4. **GitHub** account (free)
5. **Vercel**: vercel.com → **Hobby** → Continue with GitHub

B. Run on your laptop

1. Unzip → VS Code → **File › Open Folder** → `cosmotrekk`
2. **Terminal › New Terminal**
3. `node -v` (needs v20+)
4. `npm install` (1–3 min, once)
5. `npm run dev` → open **http://localhost:5173**
6. Checks: `npm test` · `npm run build`

Terminal alternative for GitHub

```
git init
git add .
git commit -m "First version of CosmoTrekk"
git branch -M main
git remote add origin
https://github.com/USER/cosmotrekk.git
git push -u origin main
```

C. Publish to GitHub (easy way)

1. Left bar → **Source Control** icon
2. **Initialize Repository**
3. Message *"First version"* → ✔ **Commit** (Yes to stage all)
4. **Publish Branch** → Publish to GitHub → name `cosmotrekk`
5. Sign in to GitHub in the browser → Allow

Shared laptop? Check the **Accounts** icon (bottom-left) shows the right GitHub user first.

D. Deploy on Vercel

1. vercel.com/new → **Import** `cosmotrekk`
2. Preset: **Vite** · Build: `npm run build` · Output: `dist`
3. **Deploy** → free link `cosmotrekk-xyz.vercel.app`
4. Every push redeploys automatically

Install as an app (PWA)
Android/desktop Chrome or Edge: menu → **Install app**.
iPhone Safari: Share → **Add to Home Screen**.
It's a web app, not an App Store app. After one visit, missions and questions work **offline**.

Optional keys: Vercel → Settings → Environment Variables (see `API_SETUP.md`). Never put secrets in `VITE_` variables.

12 The 3-minute demo script

Before going on stage: ☐ open the Vercel link (and install the PWA as a backup) ☐ Settings → **Demo mode ON** (shorter exploration; still 5 questions and 60%) ☐ speakers on ☐ a second tab with `npm run preview` in case Wi-Fi fails ☐ know one question you'll get wrong on purpose.

Time	Do	Say
0:00	Title screen	"This is **CosmoTrekk — Your Journey Has No Horizon.** A Junior Astronaut Mission Trainer."
0:15	Start Journey → name → 🎲 Randomize → Launch	"No email needed. Every look has the same abilities. **Space Walk starts at 0 km.**"
0:35	Mission map → **Moon Expedition, Tier 1**	"Start anywhere, not only Earth. Every category has the **same reward schedule**."
0:50	Briefing → point at table, diagram, sources → 🔊 3 s	"A sourced briefing pauses the game. Real distances name their reference point, and diagrams say 'not to scale'."
1:10	Begin Mission → collect items → E at the target	"Collect resources to unlock repairs. Oxygen, fuel, health, suit. It's forgiving and pauses on demand."
1:35	Answer 5 questions (one wrong on purpose)	"Three correct for clearance. A wrong answer shows the explanation and source, and the attempt carries on."
2:05	Results → **+100 km** → pick an upgrade	"Km is paid only after exploration *and* quiz, **once per milestone**, so replays can't farm it."
2:20	Know Your Space → Comets → Comparisons tab	"Three reading levels, comparisons like meteoroid vs meteor vs meteorite, practice quizzes. Studying never gives km."
2:35	Rankings	"Local scores are labelled 'On this device' and 'Demo player'. Optional Supabase mode is verified by the server."
2:45	(Optional) `/api/health`	"Gemini drafts candidate questions only from approved NASA facts. A human reviews them, and AI never grades players. It all works **offline**."
3:00	Back to title	"CosmoTrekk. **Your Journey Has No Horizon.**"

If something goes wrong on stage

Problem	Quick fix
No internet	Use the installed PWA or `npm run preview`. Gameplay is fully offline.
No sound	Click once (browsers block audio until you interact). Settings → Sound on.
Oxygen ran out mid-demo	Say "it's forgiving" → **Restart activity**. Progress is safe.
Failed the quiz by accident	Great moment: show **Review Answers** + **Retry Checkpoint** (fresh questions).

13 Judges' questions — ready answers

Question	Answer
Is the science accurate?	Facts come from a **source registry** of NASA pages (`SCIENCE_SOURCES.md`), with approximate values and named reference points. Fiction is labelled *Simulation Fiction*.
Is Space Walk real distance?	No. It's *"Simulation kilometres, not real travel distance."* The same 100/250/500/1000 schedule applies to every destination.
Can players cheat the score?	Locally, the ledger pays each milestone once and recalculates on load. In connected mode, the server grades with a private key inside a transaction. No browser game is fully cheat-proof, and we say so.
How is AI used?	Gemini only drafts **candidate** questions from approved statements. Each one is validated, then waits for human review. Grading is plain deterministic code.
What if the API fails?	Missing key, timeout, quota, bad JSON or an empty response all fall back to the **120 reviewed offline questions**.
Does it work offline / on phones?	Yes. It's a PWA with a service-worker cache and touch controls. Tested on a desktop size and a phone size.
Accessibility?	Keyboard menus, visible focus, ✓/× symbols beside colours, large buttons, reduced motion, **untimed quizzes by default**, text for all narration.
How did you test it?	17 automatic tests (3/5 passes, 2/5 fails, locking, retries, once-only rewards, tampered saves, content checks, AI validator, ranking order) plus browser runs through the full campaign and all 8 expeditions.
Cost?	₹0. Free tools and free tiers only. Optional services can stay off.
What's next?	More tiers per destination, teacher review for AI candidates, more languages. Only the 8 expeditions + campaign are playable today, and we don't advertise "infinite" missions.

Mini glossary

Component: reusable UI piece (React).
State: data that, when changed, updates the screen.
Scene: a Phaser "level" with create() and update().
Event bridge: message system between React and Phaser.
Idempotent: safe to repeat, no double effect.

PWA: website that installs and works offline.
Service worker: background script that caches files.
Serverless function: server code that runs on demand.
Row-level security: database rules for who can read or write each row.
Schema: a description of the shape data must have.

Team ownership (`TEAM_TASKS.md`): ① game scenes & controls ② UI, avatar, visuals, audio, accessibility ③ science content & questions ④ APIs, saves, backend ⑤ QA, deployment, packaging, presentation.